Problem A. Domains

Description

Vivy finds that some domains can be visited faster using different VPNs (Virtual Private Network). So she decides to set up a system to automatically transfer requests to different VPNs.

First of all, she has to set up configurations.

There are multiple rules in the configurations. The format of a rule is: [domain pattern] [VPN name]. If a domain matches the [domain pattern] of a rule, we say that the domain matches the rule, and all visits to this domain should be transferred to [VPN name].

To make the configurations short, Vivy decided to use two kinds of wildcards which are "*" and "#". Both "*" and "#" matches any strings consisting of lowercase alphabets, digits, dots and hyphens (‘-‘). But a domain matches the rules with "#" wildcard only if it doesn't match any other rules.

A domain pattern is legal, if and only if it satisfies all the following conditions:

1. The domain pattern consists of wildcards, lowercase alphabets, digits, dots and hyphens.

2. The domain pattern does not start with a dot, and does not end with a dot.

3. Any two dots in the domain pattern are not adjacent.

4. If there is a wildcard, the wildcard must be at the beginning of the domain pattern. And the wildcard must precede a “.”, or no character follows it.

5. A hyphen in the domain pattern must be between two lowercase alphabets.

A VPN name is legal, if and only if it satisfies all the following conditions:

1. The VPN name consists of lowercase alphabets, digits and hyphens.

2. The VPN name starts with lowercase alphabets.

3. A hyphen in the VPN name must be between two lowercase alphabets.

Two rules are considered conflict, if it satisfies any one of the following conditions:

1. There exist a domain without wildcards which can match both of them at the same time. For example, if the domain pattern of rule A is "*.cn" and the domain pattern of rule B is "a.cn", then rule A and rule B are conflict because domain “a.cn” can match both of them.

2. One of them can never be matched because of the other one. For example, "#.pku.cn" can never be matched if the other one is "*.cn".

A rule is considered invalid, if it satisfies any one of the following conditions:

1. The has an illegal domain pattern.

2. The rule has an illegal VPN name.

3. The rule is conflict with a valid rule that appears before it.

Now she wants to write a program to parse the configurations and determine how many rules are invalid.

And she wants to know more things. A valid domain pattern without wildcard "#" can represent a set of domains. She wants to know, given a set of domains, how many valid rules matches the set, ie. how many rules satisfy that there exists a domain without wildcards in the

set which matches the rule.

Input

The input consists of multiple test cases, up to 10.

For each test case:

The first line contains an integer N, indicating the number of rules. (1 <= N <= 50000)

Then N lines followed, each line contains two strings separated by a space, indicating domain pattern and VPN name of a rule.

The total length of the strings in rules will not exceed 1100000.

The next line contains an integer M, indicating the number of queries. (1 <= M <= 50000)

Then M lines followed. Each line contains a domain pattern representing a set of domains. For each query, you should answer how many valid rules matches the given set. It is guaranteed that the given domain pattern is legal, and does not contain wildcard "#".

The total length of the strings in queries will not exceed 1100000.

Output

For each test case:

The first line contains an integer, indicating the number of invalid rules.

Then M lines followed, each line is an integer, indicating the number of matched valid rules for the corresponding query.

Sample Input

```
11
a.c a.c
poj.org vpn-china
www.pku.edu.cn vpn-pku1
mail.pku.edu.cn vpn-pku1
*.test.pku.edu.cn vpn-pku2
*.test.pku.edu.cn vpn-pku2
*.a.test.pku.edu.cn vpn-pku2
*.test2.pku.edu.cn vpn-pku2
*.pku.edu.cn vpn-pku2
#.pku.edu.cn vpn-pku0
#.www.pku.edu.cn vpn-pku0
5
a.c
mail.pku.edu.cn
*.test.pku.edu.cn
*.pku.edu.cn
*
```

Sample Output

```
5
0
1
1
5
6
```

Explaination

There are five invalid rules in total:

1. "a.c a.c" is invalid because "a.c" is not a legal VPN name.
2. The second "*.test.pku.edu.cn vpn-pku" is invalid because it is conflict with "*.test.pku.edu.cn".
3. "*.a.test.pku.edu.cn vpn-pku" is invalid because it is conflict with "*.test.pku.edu.cn".
4. "*.pku.edu.cn vpn-pku" is invalid because it is conflict with many valid rules.
5. "#.www.pku.edu.cn vpn-pku0" is invalid because it is conflict with "#.pku.edu.cn".

For the queries:

1. "a.c" matches no rule.
2. "mail.pku.edu.cn" does not match "#.pku.edu.cn" because it matches "mail.pku.edu.cn".
3. "*.test.pku.edu.cn" only matches "*.test.pku.edu.cn".
4. "*.pku.edu.cn" may matches all valid rules except "poj.org". More specifically, "*.pku.edu.cn" contains "www.pku.edu.cn", "mail.pku.edu.cn", "a.test.pku.edu.cn", "a.test2.pku.edu.cn" and "a.pku.edu.cn". And they matches "www.pku.edu.cn", "mail.pku.edu.cn", "*.test.pku.edu.cn", "*.test2.pku.edu.cn" and "#.pku.edu.cn" correspondingly.
5. "*" may matches all valid rules.

Problem B. K-Dimensional Foil

Description

"It's too quiet," David Jr murmured to himself. His space fleet entered the battlefield several minutes ago, but there was no any trace of the enemy. Everyone on the spaceship was waiting for enemy's attack with great pressure.

Suddenly, the alarm sounded. "Wait, it's not the enemy." David realized, "it is the instrument fault." The reason for instrument fault was found out quickly: The distance between each spaceship calculated by 3-dimensional coordinates of spaceships didn't correspond with the distance calculated by communication delay. "Every ship gives the same warning, indicating that is not just a simple instrument fault. Well, it can only be explained by 'that thing.'" David believed that enemy had invented "K-Dimensional Foil", the Ultimate Weapon, and they were just being attacked by that weapon.

"K-Dimensional Foil" was a dimensional weapon. It could ascend a region in 3-dimensional space to K-dimensional (K >= 3) space, which might cause great damage to objects in that region. When a spaceship was in K-dimensional space, its coordinate was a K-dimensional coordinate. But its computer only knew the original 3-dimensional coordinate, it could not get the coordinates in other higher dimensions. So, the distance between spaceships calculated by coordinates were not correct. But the actual distances could always be calculated by communication delay.

David was a cautious man. He had prepared some way to defend the "K-dimensional Foil". However, he had to know the number of dimensions of the space in which his fleet was. This glorious mission was given to you, the chief technology officer in this fleet.

Now, it was your time to make history ……

Input

The first line of the input is an integer T (T <= 100), the number of test cases.

For each test case, the first line contains one integer n (1 <= n <= 50), the number of spaceships in David's fleet.

Then n lines follow. Each line contains three integers x, y, z (-300 <= x, y, z <= 300), the 3-dimentional coordinate of a spaceship.

Then n-1 lines follow. The ith line contains n - i positive integers. The jth integer of the ith line is $D_{i,i+j}$ $(0 \le D_{i,i+j} \le 5000000)$, which denotes the square(to avoid precision problem) of actual distance between the ith spaceship and the $(i+j)^{th}$ spaceship which is calculated by communication delay.

Notice: The distance in this problem is Euclidean distance. For example, the distance between point (x1,y1,z1,k1) and point (x2,y2,z2,k2) in 4-dimentional space is $\sqrt{(x1-x2)^2+(y1-y2)^2+(z1-z2)^2+(k1-k2)^2}$.

Output

For each test case, output an integer K, the minimal possible number of dimensions of the new space. If the input can't fit in any dimensional space, you should print "Goodbye World!", without quotations.

Sample Input

```
3
3
0 0 1
0 0 2
0 0 3
10 20
26
3
1 0 0
2 0 0
3 0 0
3 7
20
3
0 0 0
1 1 1
2 2 2
3 12
3
```

Sample Output

```
5
Goodbye World!
3
```

Hint

Case #1: In one possible situation, it's a 5-dimentional space and the coordinates of these three ships are (0,0,1,0,0), (0,0,2,0,3), (0,0,3,4,0)

Case #2: The distances between ship 1 and ship 2, ship 1 and ship 3 are both very small, but the distance between ship 2 and ship 3 is too large. That means the world is broken, so we should print "Goodbye World!"

Case #3: It seems that nothing happened.

Problem C. Graph

Description

The country contains N cities numbered from 1 to N and M undirected roads connecting pairs of cities. There are some queries. Each query is represented by two numbers: L and R , meaning that all the cities whose number is between L and R(L and R are included) are safe, and other cities are not safe. We define city A can reach city B if and only if they are both safe and there exists a path from A to B that the cities on the path are all safe.

For each query, you need to figure out the number of pairs of cities that can reach each other under the condition given by the query.

Input

First line contains one number T which means the number of test cases.

For each test case, first line contains three numbers, above mentioned N , M and Q.

Next M lines, each line contains two integers: X, Y (X != Y) which means there is a road between city X and city Y (1 <= X,Y <= N).

Next Q lines, each line contains two numbers: L, R which indicates an query(1 <= L,R <= N, L <= R).

T <= 5, N , M <= 50000, Q <= 100000

Output

For each test case, output Q lines, each line contains the answer of the correspondent query.

Example Input

```
1
6 6 4
1 2
2 3
2 6
1 5
2 4
4 5
1 4
3 6
2 6
3 4
```

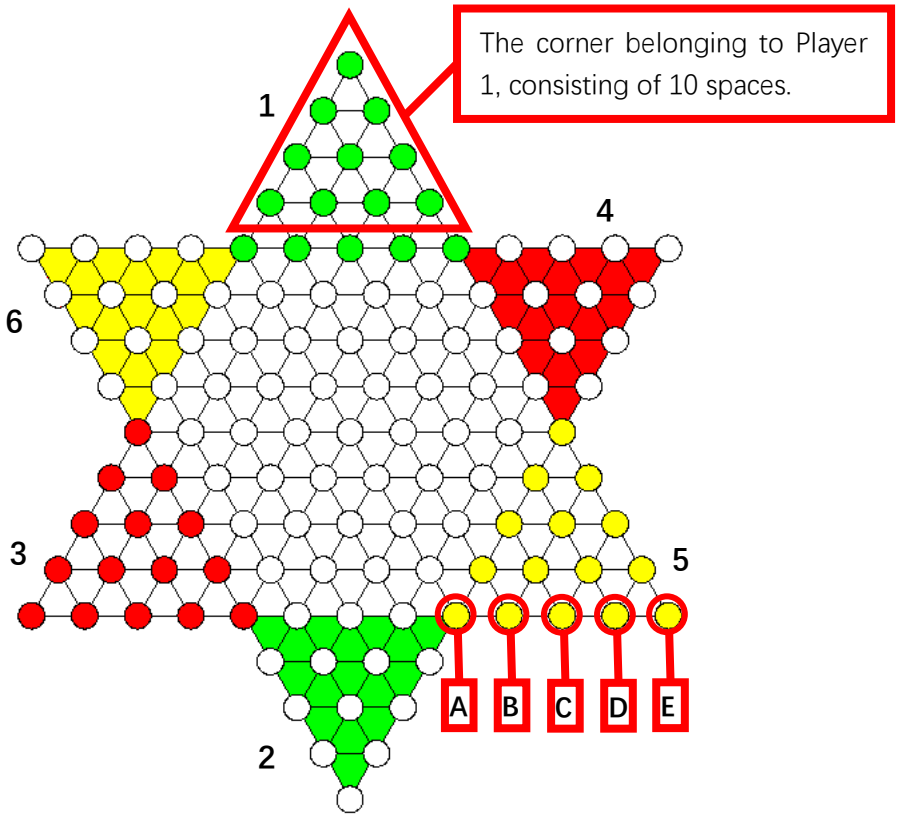
Example Output

```
6
1
10
0
```

Problem D. Chinese Checkers

Description

Chinese checkers is a strategy board game, which can be played by two, three, four, or six people, playing individually or with partners.



Six players are playing a new version of Chinese checkers, whose rules are different from the traditional ones.

1. The figure shows the shape of the board and the corner belonging to each player. A circle in the figure represents a space. Obviously, the board has 17 rows, and different rows may have different number of spaces. The corner belonging to each player consists of 10 spaces.
2. Each player has 10 pieces. At the beginning of the game, each player puts all his/her pieces on his/her corner and each piece occupies a space.
3. Players take turns moving a single piece, either by moving one step to an adjacent empty space, or by jumping over a symmetrical pattern (which includes at least one piece) to an empty space in one of six directions (left, right, upper left, upper right, lower left, lower right).

   For example, when E is an empty space, a piece can jump from A to E if and only if:

   1) there is a piece at C while B and D are empty spaces,
   2) or there are two pieces at B and D while C is an empty space,
   3) or there are three pieces at B, C and D.

   When D is an empty space, a piece can jump from A to D if and only if there are two pieces at B and C.

4. Player 1 take action first, and then Player 2, and so on.
5. A player can never move his pieces into others' corner except the corner opposite to his corner. For example, Player 6 can move his pieces into his own corner or Player 5's corner.

When the six players take their turns, they choose a piece and a direction, and then they always move the piece to the farthest possible space. Given the pieces and the directions that they choose during the first N turns, can you tell where their pieces located?

Input

The input consists of multiple test cases. (Up to 1000)

For each test case:

The first line contains an integer N, indicating the number of turns. $(1 \leq \sum N \leq 10^5)$

Then N lines follow, each line contains two integers x,y and a direction, separated by white spaces. These indicate that a player choose the piece located at the yth space of the xth row, and moves it in that direction.

For simplicity, the directions are given in form of abbreviations. **L** represents left. **R** represents right. **UL** represents upper left. **UR** represents upper right. **LL** represents lower left. **LR** represents lower right.

If the player does not have a piece located at the yth space of the xth row, or the piece can never move in the given direction, please skip his/her turn (This player do nothing in his turn). It is guaranteed that the given position and direction are valid.

Output

For each test case, output six lines. The ith line contains twenty integers $x_1, y_1, x_2, y_2, \ldots, x_{10}, y_{10}$ separated by white spaces, indicating the position of the Player i's pieces. The jth piece located at the y_j^{th} space of the x_j^{th} row. The positions should be sorted by x first and then by y in ascending order.

Sample Input

```
12
1 1 LL
16 1 UL
10 1 R
5 11 L
11 10 L
7 1 LR
2 2 LR
13 5 UL
12 2 R
5 12 L
11 11 L
6 1 LR
```

Sample Output

2 1 3 1 3 2 3 3 4 1 4 2 4 3 4 4 5 5 6 9
12 4 14 1 14 2 14 3 14 4 15 1 15 2 15 3 16 2 17 1
10 2 11 1 11 2 12 1 12 3 12 5 13 1 13 2 13 3 13 4
5 7 5 9 5 10 5 13 6 10 6 11 6 12 7 10 7 11 8 10
10 10 11 7 11 9 12 10 12 11 12 12 13 10 13 11 13 12 13 13
5 1 5 2 5 3 5 4 6 2 6 3 7 1 7 2 8 1 11 3

Problem E. Cats and Fish

Description

There are many homeless cats in PKU campus. They are all happy because the students in the cat club of PKU take good care of them. Li lei is one of the members of the cat club. He loves those cats very much. Last week, he won a scholarship and he wanted to share his pleasure with cats. So he bought some really tasty fish to feed them, and watched them eating with great pleasure. At the same time, he found an interesting question:

There are m fish and n cats, and it takes c_i minutes for the ith cat to eat out one fish. A cat starts to eat another fish (if it can get one) immediately after it has finished one fish. A cat never shares its fish with other cats. When there are not enough fish left, the cat which eats quicker has higher priority to get a fish than the cat which eats slower. All cats start eating at the same time. Li Lei wanted to know, after x minutes, how many fish would be left.

Input

There are no more than 20 test cases.

For each test case:

The first line contains 3 integers: above mentioned m, n and x (0 < m <= 5000, 1 <= n <= 100, 0 <= x <= 1000).

The second line contains n integers $c_1, c_2 \ldots c_n$, c_i means that it takes the ith cat c_i minutes to eat out a fish (1<= c_i <= 2000).

Output

For each test case, print 2 integers p and q, meaning that there are p complete fish(whole fish) and q incomplete fish left after x minutes.

Sample input

```
2 1 1
1
8 3 5
1 3 4
4 5 1
5 4 3 2 1
```

Sample output

```
1 0
0 1
0 3
```

Problem F. Secret Poems

Description

The Yongzheng Emperor (13 December 1678 – 8 October 1735), was the fifth emperor of the Qing dynasty of China. He was a very hard-working ruler. He cracked down on corruption and his reign was known for being despotic, efficient, and vigorous.

Yongzheng couldn't tolerate people saying bad words about Qing or him. So he started a movement called "words prison". "Words prison" means literary inquisition. In the famous Zhuang Tinglong Case, more than 70 people were executed in three years because of the publication of an unauthorized history of the Ming dynasty.

In short, people under Yongzheng's reign should be very careful if they wanted to write something. So some poets wrote poems in a very odd way that only people in their friends circle could read. This kind of poems were called secret poems.

A secret poem is a N×N matrix of characters which looks like random and meaning nothing. But if you read the characters in a certain order, you will understand it. The order is shown in figure 1 below:



figure 1

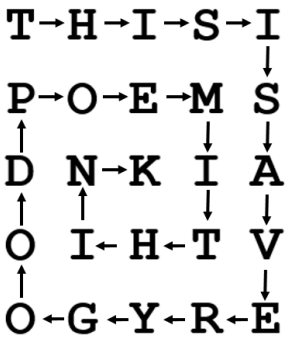


figure 2

Following the order indicated by arrows, you can get "THISISAVERYGOODPOEMITHINK", and that can mean something.

But after some time, poets found out that some Yongzheng's secret agent called "Mr. blood dripping" could read this kind of poems too. That was dangerous. So they introduced a new order of writing poems as shown in figure 2. And they wanted to convert the old poems written in old order as figure1 into the ones in new order. Please help them.

Input

There are no more than 10 test cases.

For each test case:

The first line is an integer N(1 <= N <= 100), indicating that a poem is a N×N matrix which consist of capital letters.

Then N lines follow, each line is an N letters string. These N lines represent a poem in old order.

Output

For each test case, convert the poem in old order into a poem in new order.

Sample Input

```
5
THSAD
IIVOP
SEOOH
RGETI
YMINK
2
AB
CD
4
ABCD
EFGH
IJKL
MNOP
```

Sample Output

```
THISI
POEMS
DNKIA
OIHTV
OGYRE
AB
DC
ABEI
KHLF
NPOC
MJGD
```

Problem G. Liaoning Ship's Voyage

Description

Liaoning ship, which named after a province of China, is the first aircraft carrier commissioned into the People's Liberation Army Navy. It was bought from Ukraine as a stripped hulk and was rebuilt by China as an important part of China's blue water Navy plan. Liaoning ship has sailed far into the Pacific Ocean for serval times, which shows the power and resolve of China to defend her integrity of territory.

Now Liaoning ship is on a new voyage to the Atlantic Ocean for a maneuver! The vast maneuver region on the ocean can be seen as an n×n grid which has n×n crosspoints. Each crosspoint stands for a check point of the maneuver region. Liaoning starts from the bottom-left check point whose coordinate is (0,0), and its destination is the upper-right checkpoint whose coordinate is (n-1,n-1). The positive side of the x axis points to the right, and the positive side of the y axis points up. All check points' coordinates are integral. During each move, Liaoning can go from one check point to its adjacent 8 check points **along a straight line**, and each move takes Liaoning one day. Some check points are not available to go due to the bad weather. And, as you know, on the Atlantic Ocean, there is a Bermuda Triangle in which many ships and planes were missing. Liaoning can't take risk to go into that triangle. Of course, Liaoning can't go outside the maneuver region. Please figure out a route for Liaoning to reach its destination as soon as possible.

Input

There are no more than 30 test cases.

For each case:

The first line is an integer n(2 <= n <= 20), meaning that the maneuver region is an n×n grid.

The seconds line contains six float numbers x1,y1,x2,y2,x3,y3(-100 <= x1,y1,x2,y2,x3,y3 <= 100) which have at most 2 digits after the decimal point, indicating the coordinates of the vertices of Bermuda Triangle are (x1,y1), (x2,y2) and (x3,y3). Liaoning ship can't go into that triangle, but going along its edges or touching its vertices are allowed.

Then an n×n character matrix consists of '.' and '#' follows, indicating the weather condition of all check points. '.' Means good weather and '#' means bad weather. The bottom-left character stands for the weather of check point (0,0), the character on the right side of it stands for (1,0), and the upper-right character stands for (n-1,n-1).

It is guaranteed that check point (0,0) is not inside the Bermuda Triangle or on the edges of the triangle.

Output

For each test case, print the minimum days Liaoning ship needs to reach its destination. If it is impossible for Liaoning to reach its destination, print -1 instead.

Sample Input

```
3
0.5 1.5 1.5 1.5 1 0.5
.#.
...
..#
3
0.5 1.5 1.5 1.5 1 0.5
.#.
..#
..#
```

Sample Output

```
3
-1
```

Problem H. Puzzle Game

Description

Once upon a time, there was a little dog YK. One day, he went to an antique shop and was impressed by a beautiful picture. YK loved it very much. However, YK did not have money to buy it. He begged the shopkeeper Bob whether he could have it without spending money.

Fortunately, Bob enjoyed puzzle game. He promised to give YK a picture for free if YK can solve a puzzle game for him.

Bob drew a puzzle board, which was a n×m matrix consisting of n×m cells. And there was an integer in each cell. A sub-matrix was a matrix that was a continuous part of the puzzle board (The whole puzzle board could also be called a sub-matrix). The value of a sub-matrix meant the sum of integers in the cells of the sub-matrix. The sub-matrix which had the largest value was called “the largest sub-matrix”. Bob wanted to make the value of the largest sub-matrix as small as possible by selecting one cell on the board and changed the integer in it into P. But if making that kind of change would not do anything good, he didn’t have to change any cell.

In such a situation, YK needed to calculate the minimum value of the largest sub-matrix Bob could get.

Input

There are no more than 120 test cases, but at most 3 test cases in which n >= 50 or m >=50.

The first line of each case contains three integers, above mentioned n, m and P (1<=n,m<=150,-1000<=P<=1000).

Then n lines follow. Each line contains m integers $x_1,x_2…x_m$(-1000<=x_i <=1000, i = 1…m). These n lines are the n×m integers in the n×m cells of the puzzle board.

Output

For each test case, you should output the minimum value of the largest sub-matrix Bob could get.

Sample Input

```
3 3 -10
-100 4 4
4 -10 4
4 4 1
3 3 -1
-2 -2 -2
-2 -2 -2
-2 -2 -2
```

Sample Output

```
8
-2
```

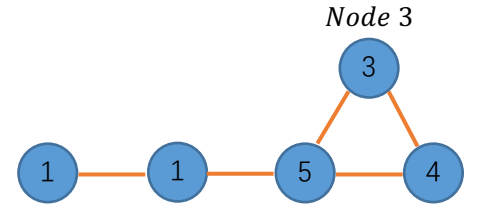
Problem I. Colored Nodes

Description

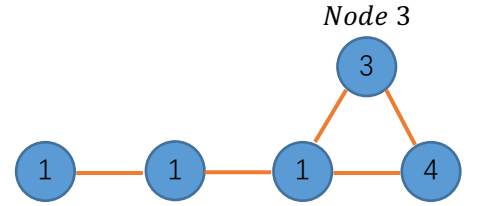
There are N colored nodes, numbered from 1 to N. Initially, the color of $Node\ i$ is $Color\ i$. There are M edges between some pairs of nodes.

At the beginning of the $(i+kN)^{th}$ minute, the colors of the nodes that adjacent to $Node\ i$ will be set to the same color as $Node\ i$, where k is a non-negative integer.

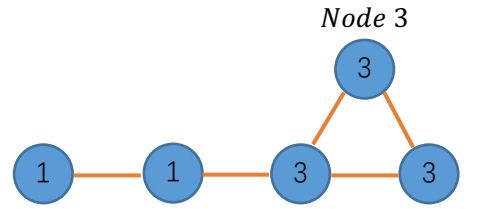
The following pictures are examples.



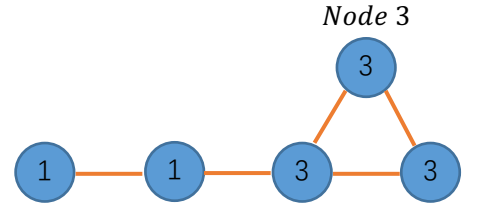
The 1st minute



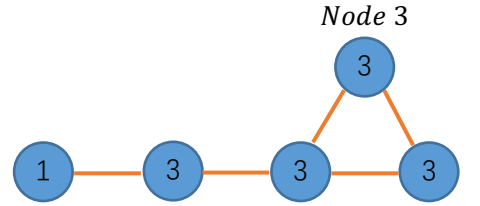
The 2nd minute



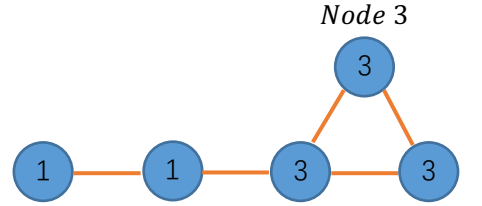
The 3rd minute



The 4th minute



The 5th minute



The 6th minute

At the beginning of the 1st minute, the color of $Node\ 2$ is set to 1.
At the beginning of the 2nd minute, the color of $Node\ 1$ and 5 is set to 1.
At the beginning of the 3rd minute, the color of $Node\ 4$ and 5 is set to 3.
At the beginning of the 4th minute, the color of $Node\ 3$ and 5 is set to 3.
At the beginning of the 5th minute, the color of $Node\ 2$, 3 and 4 is set to 3.
At the beginning of the 6th minute, the color of $Node\ 2$ is set to 1.
……

Assume the number of nodes that are color i during the jth minute is $D(i,j)$. And assume

$$F(i) = \lim_{n\to\infty} \frac{1}{n} \sum_{j=1}^{n} D(i,j)$$

Your task is to calculate $F(i)$ for all i range from 1 to N.

Input

The input consists of multiple test cases. (Up to 20)

For each test case:

The first line contains two integers N and M, indicating the number of nodes and the number of edges. $(1 \le N, M \le 10^5)$

M lines follow, each line contains two integers a and b, indicating that there is an edge between $Node\ a$ and $Node\ b$. $(1 \le a, b \le N)$

Output

For each test case, output all $F(i)$ that satisfy $F(i) \neq 0$, rounded to six decimal places and sorted in descending order.

Sample Input

5 5
1 2
2 5
3 4
4 5
3 5
5 4
1 2
2 5
5 4
4 3

Sample Output

3.000000
2.000000
2.800000
2.200000

Problem J. Pangu and Stones

Description

In Chinese mythology, Pangu is the first living being and the creator of the sky and the earth. He woke up from an egg and split the egg into two parts: the sky and the earth.

At the beginning, there was no mountain on the earth, only stones all over the land.

There were N piles of stones, numbered from 1 to N. Pangu wanted to merge all of them into one pile to build a great mountain. If the sum of stones of some piles was S, Pangu would need S seconds to pile them into one pile, and there would be S stones in the new pile.

Unfortunately, every time Pangu could only merge successive piles into one pile. And the number of piles he merged shouldn't be less than L or greater than R.

Pangu wanted to finish this as soon as possible.

Can you help him? If there was no solution, you should answer '0'.

Input

There are multiple test cases.

The first line of each case contains three integers N,L,R as above mentioned (2<=N<=100,2<=L<=R<=N).

The second line of each case contains N integers a_1,a_2 ...a_N (1<= a_i <=1000,i= 1...N), indicating the number of stones of pile 1, pile 2 ...pile N.

The number of test cases is less than 110 and there are at most 5 test cases in which N >= 50.

Output

For each test case, you should output the minimum time(in seconds) Pangu had to take . If it was impossible for Pangu to do his job, you should output 0.

Sample Input

3 2 2
1 2 3
3 2 3
1 2 3
4 3 3
1 2 3 4

Sample Output

9
6
0